

实验一：LLM 客户端开发与调用

实验一：LLM 客户端开发与调用

实验目的

1. 掌握使用 Python 编写 LLM（大语言模型）API 客户端的基本方法
2. 理解 HTTP 请求与响应的处理流程
3. 学习环境变量配置与管理
4. 实现 Token 统计功能，了解 LLM 调用的资源消耗
5. 掌握聊天历史管理机制

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

llm_client.py - 基础 LLM 客户端 - `load_env()`: 加载环境变量配置文件 - `get_token_count()`: 根据模型类型估算 Token 数量 - `call_llm()`: 调用 LLM API 并返回结果 - `main()`: 主函数，提供交互式命令行界面

chat_client.py - 带聊天历史的 LLM 客户端 - `load_env()`: 加载环境变量配置文件 - `estimate_tokens()`: 估算 Token 数量 - `call_llm()`: 调用 LLM API 并返回结果（支持流式输出参数） - `main()`: 主函数，支持多轮对话

2. 核心技术实现 环境变量管理

```
def load_env(env_path):
    env_vars = {}
    if os.path.exists(env_path):
        with open(env_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith('#') and '=' in line:
                    key, value = line.split('=', 1)
```

```
env_vars[key.strip()] = value.strip().strip(' ').strip('"')
return env_vars
```

HTTP 请求实现使用 Python 内置的 `http.client` 模块建立 HTTP/HTTPS 连接，构造 JSON 格式的请求体，发送 POST 请求到 LLM API 端点。

Token 统计机制 - 优先从 API 响应的 `usage` 字段获取精确 Token 数 - 当 API 未返回 Token 信息时，采用字符数估算方法 - GPT-3.5 模型按每 3 个字符估算 1 个 Token，其他模型按每 4 个字符估算 1 个 Token

聊天历史管理 `chat_client.py` 通过维护 `chat_history` 列表实现多轮对话，每次请求时将完整历史发送给 LLM。

3. 配置要求 需要在项目根目录创建 `.env` 文件，包含以下必需变量： - `BASE_URL`: LLM API 的基础 URL - `MODEL`: 模型名称 - `API_KEY`: API 密钥

可选配置： - `TEMPERATURE`: 温度参数（默认 0.7） - `MAX_TOKENS`: 最大生成 Token 数（默认 1000）

实验结果

功能验证

功能	llm_client.py	chat_client.py
环境变量加载	正确	正确
HTTP/HTTPS 请求	正确	正确
Token 统计	正确	正确
错误处理	正确	正确
聊天历史管理	错误	正确
流式输出支持	错误	正确

输出示例 运行 `chat_client.py` 的典型输出：

```
=====
LLM Token统计客户端
=====
API基础URL: https://api.example.com/v1
模型名称: gpt-3.5-turbo
温度参数: 0.7
最大令牌数: 1000
=====

请输入消息（输入 'exit' 或 'quit' 退出）:
> 你好
```

正在发送请求...

请求统计报告

请求耗时：0.8562 秒
输入令牌数 (Prompt): 6
输出令牌数 (Completion): 15
总令牌数: 21
处理速度: 24.52 tokens/s

LLM响应

你好！我是一个AI助手，有什么我可以帮助你的吗？

性能指标

指标	说明
请求耗时	从发送请求到接收响应的总时间
Prompt Token 数	输入消息消耗的 Token 数量
Completion Token 数	响应内容消耗的 Token 数量
总 Token 数	Prompt + Completion 的总和
处理速度	总 Token 数 / 请求耗时 (tokens/s)

实验总结

实验成果

- 1. 成功实现了两个 LLM 客户端程序，具备完整的 API 调用功能
- 2. 实现了灵活的环境变量配置机制，支持从外部文件读取配置
- 3. 建立了完善的 Token 统计体系，便于监控 API 调用成本
- 4. 实现了聊天历史管理，支持多轮对话场景

技术要点

- 1. **HTTP 客户端编程:** 使用 `http.client` 模块实现底层 HTTP 通信，理解请求头、请求体、响应解析的完整流程
- 2. **错误处理机制:** 涵盖网络异常、HTTP 错误状态码、API 响应格式异常等多种错误场景

3. **Token 估算算法**: 根据不同模型特点采用不同的字符-Token 换算比例, 提高估算准确性
4. **聊天历史管理**: 通过列表数据结构维护对话上下文, 实现上下文感知的对话交互

改进建议

1. 可添加日志记录功能, 便于问题排查和性能分析
2. 可实现请求重试机制, 提高系统稳定性
3. 可添加更丰富的 API 参数支持, 如 `top_p`、`frequency_penalty` 等
4. 可实现异步调用方式, 提升并发处理能力